

FUNDAMENTOS DE LA PROGRAMACIÓN

Guía básica de uso de git

Control de versiones con Fork

Grado en Ciencia de Datos e Inteligencia Artificial · Curso 2026/2027

1. ¿Qué es git y por qué lo usamos?

Git es un sistema de control de versiones: guarda un historial de todos los cambios que haces en tu código, para que puedas volver atrás si algo se rompe, ver qué cambió y cuándo, y trabajar con copias de seguridad reales de tu proyecto sin depender de "copia_final_v2_definitiva.py".

En esta asignatura lo vamos a usar desde el Bloque 1, y es obligatorio en las tres entregas de la práctica final: cada entrega pide un repositorio git con varios commits (no uno solo al final) y mensajes descriptivos.

- Repositorio ("repo"): la carpeta de tu proyecto, con todo su historial de cambios guardado dentro.
- Commit: una "foto" del estado de tus archivos en un momento dado, con un mensaje que explica qué cambiaste.
- Stage ("preparar" los cambios): elegir qué cambios va a incluir el próximo commit.
- Remoto (remote): una copia del repositorio alojada en internet (por ejemplo en GitHub), para tener una copia de seguridad y poder compartirlo.
- Push / Pull: subir tus commits al remoto (push) o traer los commits que hay en el remoto a tu ordenador (pull).

Nota

Git es la herramienta que hace todo esto "por dentro". Fork es el programa con ventanas y botones que usaremos para manejar git sin escribir comandos en la terminal.

2. ¿Por qué Fork?

Fork es un cliente gráfico de git: te muestra el historial, los cambios y los botones de commit/push/pull en una interfaz visual, en lugar de tener que escribir comandos. Lo recomendamos como programa oficial del curso porque:

- Tiene instalador nativo para Windows y para macOS, así que sirve independientemente del sistema operativo que uses en clase o en casa.
- Es gratuito para uso no comercial (que es nuestro caso).
- Muestra el historial de commits de forma muy visual, lo cual ayuda mucho cuando estás empezando.
- Se integra bien con GitHub, que es donde alojaremos los repositorios de la práctica.

Atención

Fork no tiene instalador oficial para Linux. Si trabajas en Linux, instala git desde la terminal (sudo apt install git en Debian/Ubuntu) y utiliza un cliente gráfico alternativo como GitKraken, o el propio control de código fuente integrado en Visual Studio Code. Pregunta en clase si este es tu caso.

3. Instalar Fork

3.1 Windows y macOS

1. Entra en <https://git-fork.com>

2. Pulsa el botón de descarga correspondiente a tu sistema operativo (Windows o macOS).
3. Ejecuta el instalador descargado y sigue los pasos por defecto.
4. La primera vez que abras Fork, te pedirá tu nombre y tu correo electrónico: usa el mismo correo con el que vayas a crear tu cuenta de GitHub, ya que es lo que identificará tus commits como tuyos.

Nota

Si Fork te pide instalar git por separado durante la instalación, acepta: Fork necesita que git esté instalado en tu sistema para funcionar (Fork es solo la interfaz visual).

3.2 Crear una cuenta en GitHub

Necesitarás una cuenta gratuita en <https://github.com> para alojar tus repositorios y entregar tu práctica. Regístrate con el mismo correo que usaste al configurar Fork, así todo queda enlazado correctamente.

4. Crear tu primer repositorio

Hay dos formas de empezar a trabajar con git en un proyecto: crear el repositorio en tu ordenador (para empezar un proyecto nuevo) o clonar uno que ya existe en GitHub (para descargarte uno que ya está creado).

4.1 Crear un repositorio nuevo desde una carpeta que ya tienes

1. Abre Fork.
2. En el menú, elige "File" → "New" (o el botón "+" de la pantalla de inicio).
3. Selecciona la carpeta donde tienes tu programa de Python.
4. Fork inicializará un repositorio git en esa carpeta (esto es exactamente lo mismo que el comando git init).

4.2 Subir tu repositorio a GitHub

1. En GitHub, pulsa "New repository", ponle un nombre (por ejemplo: ajedrez-fundamentos-programacion) y NO marques la opción de crear un README (para evitar conflictos con tu carpeta local). Crea el repositorio.
2. GitHub te mostrará una URL del tipo <https://github.com/tu-usuario/tu-repo.git> — cópiala.
3. En Fork, con tu repositorio local abierto, ve a "Repository" → "Add Remote", pega esa URL y guarda.
4. Pulsa el botón "Push" en la barra superior de Fork para subir tu historial al repositorio remoto.

Nota

Este proceso (crear el repositorio local + conectarlo a GitHub) solo se hace una vez por proyecto: lo harás en el Bloque 1 y seguirás usando el MISMO repositorio durante las tres entregas de la práctica final (Entrega 1 → Entrega 2 → Entrega 3), sin crear uno nuevo cada vez.

4.3 Clonar un repositorio que ya existe

Si en algún momento necesitas descargarte un repositorio que ya está en GitHub (por ejemplo, uno de ejemplo que comparta el profesor):

1. En Fork, ve a "File" → "Clone".
2. Pega la URL del repositorio de GitHub.
3. Elige en qué carpeta de tu ordenador quieres guardarlo.
4. Pulsa "Clone".

5. El flujo de trabajo básico: cambiar, preparar, confirmar, subir

Este es el ciclo que repetirás constantemente durante el curso, cada vez que termines algo con lo que estés satisfecho: un ejercicio, una función, una corrección de un error.

5.1 Ver qué has cambiado

Cada vez que guardas un cambio en tu archivo .py y abres Fork, verás la lista de archivos modificados en el panel "Changes" (o "Working Copy"), en la parte superior izquierda. Al hacer clic en un archivo, Fork te muestra exactamente qué líneas se han añadido (en verde) y cuáles se han eliminado (en rojo).

5.2 Preparar los cambios (stage)

Antes de hacer un commit, tienes que decidir qué cambios quieres incluir en él. En Fork esto se hace marcando (o arrastrando) los archivos desde "Unstaged" a "Staged". Al principio del curso, normalmente incluirás todos los archivos modificados en cada commit.

5.3 Confirmar los cambios (commit)

Con los archivos en "Staged", escribe un mensaje de commit en el cuadro de texto de la parte inferior izquierda y pulsa el botón "Commit". El mensaje debe describir QUÉ has hecho, no cómo te sientes ni información irrelevante.

Mensajes de commit BUENOS:

```
"Añade la función para imprimir el tablero con coordenadas"  
"Corrige el error de índice al consultar la última fila"  
"Implementa la máquina de estados de turnos"
```

Mensajes de commit MALOS:

```
"cambios"  
"asdasd"  
"funciona ya por fin"
```

Recuerda

Haz un commit por cada avance significativo y terminado (no dejes todo el trabajo del día en un único commit al final). Cuantos más commits pequeños y bien descritos tengas, más fácil será para ti (y para quien corrija) ver cómo evolucionó tu programa — y es justo lo que se pide en el criterio de entrega de cada práctica.

5.4 Subir los cambios (push)

Un commit se guarda primero solo en tu ordenador. Para que quede también en GitHub (y así entregado / respaldado), pulsa el botón "Push" en la barra superior de Fork. Hazlo al menos al final de cada sesión de trabajo.

5.5 Bajar cambios (pull)

Si trabajas desde varios ordenadores, o en el futuro colaboras con otra persona, usa el botón "Pull" para traer a tu ordenador los commits que ya están en GitHub pero no en tu copia local. Hazlo siempre al empezar a trabajar, antes de hacer cambios nuevos.

5.6 Ver el historial

El panel central de Fork muestra el árbol de commits: cada punto es un commit, con su mensaje, autor y fecha. Es el equivalente visual al comando git log, y te permite comprobar rápidamente que tienes "varios commits con mensajes descriptivos" como exige cada entrega.

6. Resumen del ciclo de trabajo

Paso	Qué haces	Botón en Fork
1	Modificas tu archivo .py en VS Code y lo guardas	—
2	Revisas qué ha cambiado	Panel "Changes"
3	Preparas los archivos que quieres confirmar	"Stage"
4	Escribes un mensaje descriptivo y confirmas	"Commit"
5	Subes tus commits a GitHub	"Push"

7. Errores frecuentes al empezar

- Olvidar hacer "Stage" antes de "Commit": si el botón de commit aparece deshabilitado, revisa que tengas al menos un archivo en la zona "Staged".
- No hacer "Push": los commits se quedan solo en tu ordenador. Si entregas y no has hecho push, en GitHub no aparecerá nada.
- Hacer un único commit gigante al final del bloque: no cumple el requisito de "un commit por avance significativo" que piden las entregas.
- Subir sin querer archivos que no son de tu programa (por ejemplo carpetas del entorno virtual): puedes crear un archivo .gitignore para excluirlos; pregunta en clase si te ocurre.
- Cambiar de ordenador y no hacer "Pull" antes de seguir trabajando: puede generar conflictos entre versiones distintas de tu propio código.

8. Relación con las entregas de la práctica

Cada una de las tres entregas de la práctica final (tras el Bloque 3, tras el Bloque 6 y tras el Bloque 8) exige explícitamente:

- Un repositorio git para el proyecto, creado en el Bloque 1 y continuado en el mismo repositorio a lo largo de las tres entregas (no uno nuevo cada vez).
- Varios commits con mensajes descriptivos que reflejen el avance del trabajo, no un único commit al final.

Recuerda

El historial de commits en Fork (panel central) es exactamente lo que evidencia este requisito: antes de entregar, revisa que tu árbol de commits cuente la historia de cómo construiste tu programa.

Si tienes cualquier problema instalando o usando Fork, pregunta en clase o en las horas de tutoría — es mejor resolver un problema de git al principio del curso que descubrirlo justo antes de una entrega.